Deploying PleIAs Models on Google Cloud Run

A Complete Guide for API Access

Target Audience: Claude Opus 4.5 developing an app via Github and Netlify
Last Updated: January 8, 2026
Estimated Setup Time: 30-45 minutes

Table of Contents

1. Overview
2. Prerequisites
3. Architecture Overview
4. Step 1: Set Up Google Cloud
5. Step 2: Create the Flask API
6. Step 3: Dockerize the Application
7. Step 4: Deploy to Cloud Run
8. Step 5: Test Your API
9. Step 6: Connect to Your Netlify App
10. Monitoring and Cost Management
11. Troubleshooting
12. Appendix: Alternative Models

Overview

What You're Building

A serverless API endpoint that:

- Hosts ethically-trained PleIAs language models
- Provides text generation capabilities via REST API
- Runs on Google Cloud Run's free tier (never expires)
- Can be accessed by your Netlify-hosted frontend app

Why This Stack?

- **PleIAs Models:** Trained exclusively on public domain/licensed data
- **Google Cloud Run:** Generous free tier (2M requests/month, 180k vCPU-seconds)
- **Flask:** Lightweight Python web framework
- **Docker:** Containerized deployment for consistency
- **Netlify:** Easy frontend hosting with GitHub integration

Free Tier Limits (What You Get Free Forever)

- **2 million requests** per month
- **180,000 vCPU-seconds** (50 hours of CPU time)
- **360,000 GB-seconds** of memory
- **1 GB** of outbound data transfer (North America)

For light usage and development, this should cover your needs indefinitely.

Prerequisites

Required Software

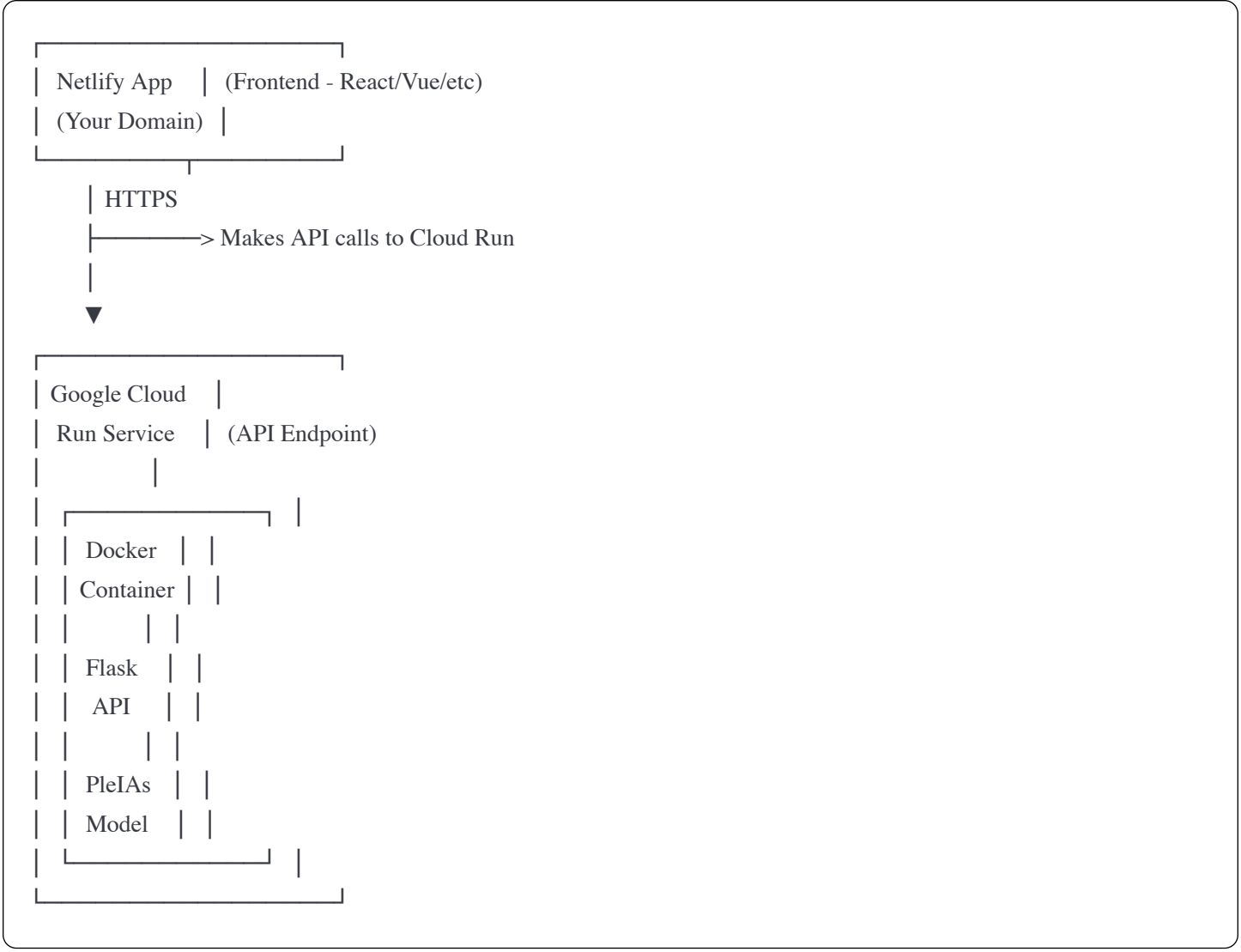
Install these tools on your development machine:

1. **Python 3.9+**
    - Check: `python3 --version`
    - Install: https://www.python.org/downloads/
2. **Docker Desktop**
    - Check: `docker --version`
    - Install: https://www.docker.com/products/docker-desktop/
3. **Google Cloud CLI (gcloud)**
    - Check: `gcloud --version`
    - Install: https://cloud.google.com/sdk/docs/install
4. **Git**
    - Check: `git --version`
    - Install: https://git-scm.com/downloads

Required Accounts

1. **Google Cloud Account**
    - Sign up: https://cloud.google.com/
    - Credit card required (for identity verification)
    - **Note:** Your $2/month Drive storage does NOT affect free tier eligibility
    - Free tier is separate and available to all accounts
2. **GitHub Account** (you already have this)
3. **Netlify Account** (you already have this)

Architecture Overview



Request Flow:

1. User interacts with your Netlify app
2. Frontend sends POST request to Cloud Run API
3. Cloud Run loads model (if cold start) and generates text
4. Response returned to frontend
5. Frontend displays result to user

Step 1: Set Up Google Cloud

1.1 Create a New Project

```bash
# Log in to Google Cloud
gcloud auth login

# Create a new project (choose a unique name)
gcloud projects create pleias-api-project --name="PleIAs API"

# Set as active project
gcloud config set project pleias-api-project

# Enable required APIs
gcloud services enable run.googleapis.com
gcloud services enable containerregistry.googleapis.com
gcloud services enable cloudbuild.googleapis.com
```

Why these APIs?

- `run.googleapis.com`: Cloud Run service
- `containerregistry.googleapis.com`: For storing Docker images
- `cloudbuild.googleapis.com`: For building containers

1.2 Set Up Billing (Required for Cloud Run)

Even though you'll stay within the free tier, you must enable billing:

1. Go to: https://console.cloud.google.com/billing
2. Link your project to a billing account
3. **Don't worry:** Free tier usage won't be charged

1.3 Configure Default Region

Choose a region close to your users (affects latency and free tier eligibility):

```bash
# Set region (use us-central1 for US free tier)
gcloud config set run/region us-central1
```

Recommended regions for free tier:

- `us-central1` (Iowa) - US
- `us-west1` (Oregon) - US West Coast
- `us-east1` (South Carolina) - US East Coast

Step 2: Create the Flask API

2.1 Project Structure

Create a new directory for your API:

```bash
mkdir pleias-api
cd pleias-api
```

Your project structure will look like this:

```
pleias-api/
├── app.py              # Flask application
├── requirements.txt    # Python dependencies
├── Dockerfile          # Container definition
└── .dockerignore       # Files to exclude from image
```

2.2 Create requirements.txt

Create a file named `requirements.txt`:

```txt
flask==3.0.0
gunicorn==21.2.0
transformers==4.36.0
torch==2.1.0
sentencepiece==0.1.99
protobuf==4.25.1
```

What each dependency does:

- `flask`: Web framework for the API
- `gunicorn`: Production-grade WSGI server
- `transformers`: Hugging Face library for loading models
- `torch`: PyTorch for model inference
- `sentencepiece`: Tokenizer for some models
- `protobuf`: Required by transformers

2.3 Create app.py

Create a file named `app.py`:

python

```python
import os
import logging
from flask import Flask, request, jsonify
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

# Configure logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

app = Flask(__name__)

# Model configuration
MODEL_NAME = "PleIAs/Pleias-350m-Preview"
model = None
tokenizer = None

def load_model():
    """Load the model and tokenizer at startup"""
    global model, tokenizer

    logger.info(f"Loading model: {MODEL_NAME}")

    try:
        # Load tokenizer
        tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

        # Load model (CPU-only for cost efficiency)
        model = AutoModelForCausalLM.from_pretrained(
            MODEL_NAME,
            torch_dtype=torch.float32,
            low_cpu_mem_usage=True
        )

        logger.info("Model loaded successfully")

    except Exception as e:
        logger.error(f"Error loading model: {str(e)}")
        raise

# Load model at startup
load_model()

@app.route('/health', methods=['GET'])
def health_check():
```

```
def health_check():
    """Health check endpoint"""
    return jsonify({
        'status': 'healthy',
        'model': MODEL_NAME,
        'model_loaded': model is not None
    })

@app.route('/generate', methods=['POST'])
def generate_text():
    """Generate text from a prompt"""

    try:
        # Get request data
        data = request.get_json()

        if not data or 'prompt' not in data:
            return jsonify({
                'error': 'Missing required field: prompt'
            }), 400

        prompt = data['prompt']
        max_length = data.get('max_length', 100)
        temperature = data.get('temperature', 0.7)

        # Validate parameters
        if max_length > 500:
            return jsonify({
                'error': 'max_length cannot exceed 500 tokens'
            }), 400

        logger.info(f"Generating text for prompt: {prompt[:50]}...")

        # Tokenize input
        inputs = tokenizer(prompt, return_tensors="pt")

        # Generate
        with torch.no_grad():
            outputs = model.generate(
                inputs.input_ids,
                max_length=max_length,
                temperature=temperature,
                do_sample=True,
                pad_token_id=tokenizer.eos_token_id
            )

        # Decode output
```

```
        generated_text = tokenizer.decode(outputs[0], skip_special_tokens=True)

        # Return response
        return jsonify({
            'prompt': prompt,
            'generated_text': generated_text,
            'model': MODEL_NAME
        })

    except Exception as e:
        logger.error(f"Error generating text: {str(e)}")
        return jsonify({
            'error': 'Internal server error',
            'details': str(e)
        }), 500

@app.route('/', methods=['GET'])
def root():
    """Root endpoint with API information"""
    return jsonify({
        'name': 'PleIAs API',
        'model': MODEL_NAME,
        'endpoints': {
            'health': '/health',
            'generate': '/generate (POST)'
        },
        'usage': {
            'generate': {
                'method': 'POST',
                'body': {
                    'prompt': 'Your text here (required)',
                    'max_length': 'Maximum tokens (default: 100)',
                    'temperature': 'Sampling temperature (default: 0.7)'
                }
            }
        }
    })

if __name__ == '__main__':
    # This is only used for local development
    # Cloud Run uses Gunicorn in production
    port = int(os.environ.get('PORT', 8080))
    app.run(host='0.0.0.0', port=port, debug=False)
```

Key features of this API:

- Health check endpoint for monitoring
- Text generation endpoint with configurable parameters
- Proper error handling and logging
- CPU-only inference (cheaper than GPU)
- Respects PORT environment variable (required by Cloud Run)

2.4 Test Locally (Optional but Recommended)

```bash
# Install dependencies
pip3 install -r requirements.txt

# Run locally
python3 app.py
```

Visit http://localhost:8080 to see API info.

Test the generate endpoint:

```bash
curl -X POST http://localhost:8080/generate \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Once upon a time", "max_length": 50}'
```

Step 3: Dockerize the Application

3.1 Create Dockerfile

Create a file named `Dockerfile`:

dockerfile

```
# Use Python 3.10 slim image
FROM python:3.10-slim

# Set working directory
WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements first (for caching)
COPY requirements.txt .

# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .

# Set environment variables
ENV PORT=8080
ENV PYTHONUNBUFFERED=1

# Expose port
EXPOSE 8080

# Use Gunicorn for production
CMD exec gunicorn --bind :$PORT --workers 1 --threads 2 --timeout 0 app:app
```

Docker configuration explained:

- `python:3.10-slim`: Lightweight base image
- `--workers 1`: Single worker process (more workers = more memory)
- `--threads 2`: Handle 2 requests concurrently
- `--timeout 0`: No timeout (model loading can be slow)

3.2 Create .dockerignore

Create a file named `.dockerignore`:

```
__pycache__/
*.pyc
*.pyo
*.pyd
.Python
env/
venv/
.git/
.gitignore
README.md
.DS_Store
```

3.3 Build Docker Image Locally (Optional)

Test your Docker build:

```bash
# Build image
docker build -t pleias-api .

# Test locally
docker run -p 8080:8080 -e PORT=8080 pleias-api
```

Visit http://localhost:8080 to verify it works.

Step 4: Deploy to Cloud Run

4.1 Build and Push to Google Container Registry

```bash
# Set project ID (replace with your project)
PROJECT_ID="pleias-api-project"

# Build and push in one command
gcloud builds submit --tag gcr.io/$PROJECT_ID/pleias-api
```

This will:

1. Build your Docker image in the cloud
2. Push it to Google Container Registry
3. Take 3-5 minutes

4.2 Deploy to Cloud Run

```bash
# Deploy to Cloud Run
gcloud run deploy pleias-api \
  --image gcr.io/$PROJECT_ID/pleias-api \
  --platform managed \
  --region us-central1 \
  --allow-unauthenticated \
  --memory 2Gi \
  --cpu 2 \
  --timeout 300 \
  --max-instances 10 \
  --min-instances 0
```

Configuration explained:

- `--allow-unauthenticated`: Public API (no auth required)
- `--memory 2Gi`: 2GB RAM (needed for model)
- `--cpu 2`: 2 vCPUs (faster inference)
- `--timeout 300`: 5-minute timeout (for slow cold starts)
- `--max-instances 10`: Scale up to 10 instances under load
- `--min-instances 0`: Scale to zero when idle (save money)

Deployment will take 2-3 minutes.

4.3 Get Your API URL

After deployment, you'll see output like:

```
Service [pleias-api] revision [pleias-api-00001-xyz] has been deployed and is serving 100 percent of traffic.
Service URL: https://pleias-api-abc123-uc.a.run.app
```

Save this URL! This is your API endpoint.

Step 5: Test Your API

5.1 Test Health Endpoint

```bash
# Replace with your actual URL
API_URL="https://pleias-api-abc123-uc.a.run.app"

curl $API_URL/health
```

Expected response:

```json
{
  "status": "healthy",
  "model": "PleIAs/Pleias-350m-Preview",
  "model_loaded": true
}
```

5.2 Test Text Generation

```bash
curl -X POST $API_URL/generate \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": "Once upon a time in a distant land",
    "max_length": 100,
    "temperature": 0.8
  }'
```

First request will be slow (30-60 seconds) - this is the "cold start" as the container loads.

Subsequent requests will be much faster (1-5 seconds).

5.3 Test from Browser

Visit your API URL in a browser to see the API documentation.

Step 6: Connect to Your Netlify App

6.1 Store API URL in Netlify Environment Variables

1. Go to your Netlify site dashboard
2. Navigate to: **Site settings → Environment variables**
3. Add a new variable:
    - **Key:** `VITE_API_URL` (or `REACT_APP_API_URL` for Create React App)
    - **Value:** Your Cloud Run URL (e.g., `https://pleias-api-abc123-uc.a.run.app`)
4. Click **Save**
5. Redeploy your site

6.2 Example Frontend Code

React/Vue/Svelte example:

javascript

```javascript
// In your frontend code
const API_URL = import.meta.env.VITE_API_URL; // Vite
// OR
const API_URL = process.env.REACT_APP_API_URL; // Create React App

async function generateText(prompt) {
  try {
    const response = await fetch(`${API_URL}/generate`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        prompt: prompt,
        max_length: 150,
        temperature: 0.7
      })
    });

    if (!response.ok) {
      throw new Error(`API error: ${response.status}`);
    }

    const data = await response.json();
    return data.generated_text;

  } catch (error) {
    console.error('Generation error:', error);
    throw error;
  }
}

// Usage
const result = await generateText("Write a story about");
console.log(result);
```

6.3 Handle Long Response Times

Since inference can take a few seconds, show loading states:

```javascript
async function handleGenerate() {
  setLoading(true);
  setError(null);

  try {
    const text = await generateText(userPrompt);
    setGeneratedText(text);
  } catch (err) {
    setError('Failed to generate text. Please try again.');
  } finally {
    setLoading(false);
  }
}
```

6.4 CORS Configuration (If Needed)

If you get CORS errors, add CORS headers to your Flask app:

```python
# Add to app.py
from flask_cors import CORS

app = Flask(__name__)
CORS(app)  # Enable CORS for all routes
```

Add to requirements.txt:

```
flask-cors==4.0.0
```

Then redeploy.

Monitoring and Cost Management

Monitor Usage

1. Go to Google Cloud Console
2. Navigate to: **Cloud Run → pleias-api → Metrics**
3. Monitor:
    - Request count
    - Container CPU utilization
    - Container memory utilization
    - Request latency

Check Costs

1. Go to: **Billing → Reports**
2. Filter by: **Cloud Run**
3. You should see $0.00 if within free tier

Set Up Budget Alerts

1. Go to: **Billing → Budgets & alerts**
2. Create budget: **$5/month**
3. Set alert at: **50%, 90%, 100%**
4. Add your email

This way, you'll be notified if you accidentally exceed free tier.

Free Tier Usage Tracking

Monthly free tier limits:

- **Requests:** 2,000,000 (you'll likely use < 10,000)
- **CPU:** 180,000 vCPU-seconds (50 hours)
- **Memory:** 360,000 GB-seconds
- **Network:** 1 GB egress

Estimated usage for typical app:

- 1,000 requests/month
- ~5 seconds per request
- ~2GB memory per instance
- **Cost:** $0.00 (well within free tier)

Troubleshooting

Issue: "Container failed to start"

Cause: Out of memory or startup timeout

Solution:

```bash
# Increase memory and timeout
gcloud run services update pleias-api \
  --memory 4Gi \
  --timeout 600
```

Issue: "Cold starts are too slow"

Cause: Container scales to zero, needs to reload model

Solutions:

Option 1: Keep one instance warm (costs ~$10/month)

```bash
gcloud run services update pleias-api --min-instances 1
```

Option 2: Use a smaller model (faster loading)

- Try `PleIAs/Monad` (56M parameters) instead of 350M

Option 3: Accept cold starts for free tier

- First request: 30-60 seconds
- Subsequent requests: 1-5 seconds
- Container stays warm for ~15 minutes after last request

Issue: "Error loading model"

Cause: Model not found or network issues

Solution:

1. Check model name in app.py
2. Verify model exists on Hugging Face
3. Check Cloud Run logs:

```bash
gcloud run services logs read pleias-api --limit 50
```

Issue: "Request timeout"

Cause: Generation taking too long

Solution:

```bash
# Increase timeout
gcloud run services update pleias-api --timeout 600

# OR reduce max_length in your requests
{
  "prompt": "...",
  "max_length": 50  // Lower this
}
```

Issue: "Out of memory"

Cause: Model too large for allocated memory

Solutions:

Option 1: Use smaller model

```python
# In app.py, change to:
MODEL_NAME = "PleIAs/Monad"  # 56M params instead of 350M
```

Option 2: Increase memory (may exceed free tier)

```bash
gcloud run services update pleias-api --memory 4Gi
```

View Logs

```bash
# Real-time logs
gcloud run services logs tail pleias-api

# Last 100 lines
gcloud run services logs read pleias-api --limit 100

# Errors only
gcloud run services logs read pleias-api --severity=ERROR
```

Appendix: Alternative Models

Available PleIAs Models

All models are trained exclusively on public domain/licensed data:

Model	Size	Best For	Memory Needed
PleIAs/Monad	56M	Ultra-fast, minimal resources	1Gi
PleIAs/Pleias-350m-Preview	350M	Balanced performance	2Gi
PleIAs/Pleias-1.2b-Preview	1.2B	Better quality	4Gi
PleIAs/Pleias-3b-Preview	3B	Best quality	8Gi+
PleIAs/Baguettotron	321M	Reasoning tasks	2Gi

To Use a Different Model

1. Edit `app.py`:

```python
# Change this line:
MODEL_NAME = "PleIAs/Monad"  # Or any other model
```

2. Rebuild and redeploy:

```bash
gcloud builds submit --tag gcr.io/$PROJECT_ID/pleias-api
gcloud run deploy pleias-api --image gcr.io/$PROJECT_ID/pleias-api
```

Recommended Model Selection

For free tier:

- **Development/Testing:** Monad (56M) - fastest, cheapest
- **Production (light use):** Pleias-350m-Preview - good balance
- **Production (better quality):** Pleias-1.2b-Preview - may need paid tier

For paid tier:

- **Best quality:** Pleias-3b-Preview (expect ~$20-50/month)

Quick Reference

Common Commands

```bash
# Deploy/update service
gcloud run deploy pleias-api --image gcr.io/$PROJECT_ID/pleias-api

# View logs
gcloud run services logs tail pleias-api

# Update memory
gcloud run services update pleias-api --memory 2Gi

# Update timeout
gcloud run services update pleias-api --timeout 300

# Delete service (cleanup)
gcloud run services delete pleias-api

# Check billing
gcloud billing accounts list
```

API Endpoints

- **Health check:** `GET /health`
- **Generate text:** `POST /generate`
- **API info:** `GET /`

Support Resources

- **Google Cloud Run Docs:** https://cloud.google.com/run/docs
- **Hugging Face Transformers:** https://huggingface.co/docs/transformers
- **PleIAs Models:** https://huggingface.co/PleIAs
- **Flask Documentation:** https://flask.palletsprojects.com/

Summary

You now have:

- ✅A serverless API hosting ethically-trained PleIAs models
- ✅Running on Google Cloud Run's free tier
- ✅Accessible via HTTPS from your Netlify app
- ✅Scalable from 0 to 10 instances automatically
- ✅Costs $0/month for typical development usage

Next Steps:

1. Share your Cloud Run API URL with your frontend developers
2. Build your Netlify app to consume the API
3. Monitor usage to stay within free tier
4. Scale up to paid tier if needed later

Questions? Check the troubleshooting section or Google Cloud documentation.

Document Version: 1.0
Last Updated: January 8, 2026
Maintained by: Your deployment team